

Day 97: Gradients and Gradient Descent — Beginner Introduction

1. Day number

Day 97

2. Topic name

Gradient and Gradient Descent

Gradient descent is a method that helps a machine-learning model **gradually move toward a lower error**.

3. Connection

Yesterday, you learned that a **derivative** tells us how a function changes with respect to one variable.

Today, we connect that idea to machine learning:

```
Derivative
  ↓
tells us the direction of change

Gradient
  ↓
tells us how the loss changes
with model parameters

Gradient descent
  ↓
uses that information to
reduce the loss step by step
```

You do not need vector calculus today. For a single parameter, you can think of the gradient almost like the derivative you learned yesterday.

4. Important topics

The key ideas are:

- **Loss/error** — how wrong the model currently is
- **Derivative** — tells how the error changes
- **Gradient** — direction of increasing error
- **Learning rate** — size of each update step
- **Iterative update** — repeatedly improving the parameter

The overall goal is:

Find parameter values that make the model's error as small as possible.

5. Foundational notes

Imagine a simple model has one adjustable value:

w = model parameter

Different values of w produce different amounts of error.

For example:

w	error
0	25
1	16
2	9
3	4
4	1
5	0
6	1
7	4

The model would prefer:

$w = 5$

because that gives the smallest error in this simplified example.

But in real ML problems, the model usually does not already know the best parameter.

It needs a method for finding it.

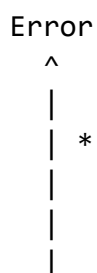
That is where **gradient descent** comes in.

6. Hill/downhill analogy

Imagine you are standing somewhere on a hill in thick fog.

You cannot see the whole landscape.

Your goal is to reach the bottom.



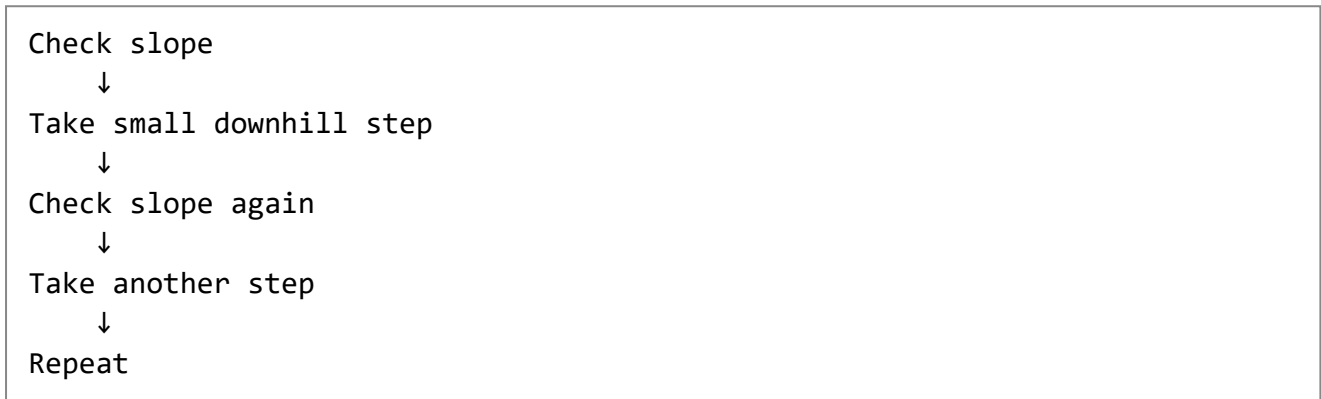


You can feel the slope beneath your feet.

If the ground rises toward your right, you move left.

If the ground rises toward your left, you move right.

Then you check the slope again.



This is the basic intuition behind **gradient descent**.

The model repeatedly asks:

| Which direction makes the error smaller?

7. What is a gradient?

Yesterday, for one variable, you learned about a derivative.

Suppose the derivative of the loss is:



That means moving the parameter to the right makes the error increase locally.

So gradient descent moves in the **opposite direction**:



If the gradient is negative:

```
negative gradient
  ↓
error falls toward right
  ↓
move right
```

If it is close to zero:

```
gradient  $\approx 0$ 
```

the surface is relatively flat, which can mean we are near a minimum.

So remember:

▮ **The gradient points uphill; gradient descent moves downhill.**

8. Learning rate

Knowing the direction is not enough.

We must also decide **how large a step to take**.

That step size is controlled by the **learning rate**.

Imagine:

```
learning rate = step size
```

A small learning rate:

```
Current position
  ↓
  ●
  → ●
    → ●
      → ●
```

means many small movements.

A large learning rate:

```
● -----> ●
```

means bigger movements.

The learning rate is often written conceptually as:

```
learning_rate
```

or with the Greek letter:

α (alpha)

You do not need to memorize the symbol yet.

9. Easy numerical example

Imagine our parameter starts at:

$w = 8$

Suppose the minimum error occurs somewhere near:

$w = 5$

At $w = 8$, imagine the gradient is positive.

That tells us:

moving right $\rightarrow$ error increases
moving left $\rightarrow$ error decreases

So we take a small step left.

Perhaps:

8
 $\downarrow$
7
 $\downarrow$
6
 $\downarrow$
5.5
 $\downarrow$
5.2
 $\downarrow$
5.05

Notice what is happening.

We are not jumping directly to the answer.

Instead:

calculate direction
 $\downarrow$
take a small step
 $\downarrow$
calculate again
 $\downarrow$

```
take another step
  ↓
repeat
```

The parameter gradually approaches the area with lower error.

10. The basic update idea

A simplified gradient-descent update is:

```
new parameter
=
old parameter - learning rate × gradient
```

Focus on the meaning rather than memorizing the formula.

Suppose:

```
old parameter = 8

learning rate = 0.1

gradient = 4
```

Then:

```
change = 0.1 × 4
        = 0.4
```

Because the gradient is positive, subtract it:

```
new parameter
= 8 - 0.4
= 7.6
```

Now calculate the gradient again at 7.6.

Perhaps the next update gives:

```
7.6
  ↓
7.25
  ↓
6.95
  ↓
...
```

The process continues toward lower error.

11. Problem statement

Imagine a model has one parameter:

w

You start at:

$w = 10$

Suppose the error becomes smaller as you move toward:

$w = 4$

At your starting position, the gradient tells you that you should move toward smaller values.

Conceptually show several repeated updates:

Start:

$w = 10$

Then think about:

Update 1 → ?

Update 2 → ?

Update 3 → ?

Update 4 → ?

Your goal is **not** to jump immediately to **4**.

Instead, show how a sequence of small steps could gradually approach the lower-error region.

12. Concepts used

You are combining several ideas:

```
Model parameter
  ↓
Prediction
  ↓
Loss / error
  ↓
Derivative or gradient
  ↓
Direction of increasing error
```

↓
Move in opposite direction
↓
Update parameter
↓
Repeat

Important vocabulary:

Loss: tells us how wrong the model is.

Gradient: tells us how the loss changes.

Learning rate: controls how big our update is.

Iteration: one round of calculating and updating.

Gradient descent: repeating those updates to reduce loss.

13. Thought process

When thinking about gradient descent, ask these questions in order:

1. What parameter am I changing?
2. How large is the current error?
3. What does the gradient say?
4. Which direction reduces error?
5. How large should my step be?
6. Update the parameter.
7. Calculate again.
8. Repeat until improvement becomes small.

Do not think:

| Find the perfect parameter immediately.

Think:

| Improve the parameter a little, then repeat.

That repeated improvement is a major idea in machine learning.

14. Beginner-friendly pseudocode

A simplified version looks like this:

```
choose starting parameter

choose learning rate

repeat:
    calculate model error

    calculate gradient

    parameter =
        parameter - learning_rate * gradient

    check the new error
```

Conceptually:

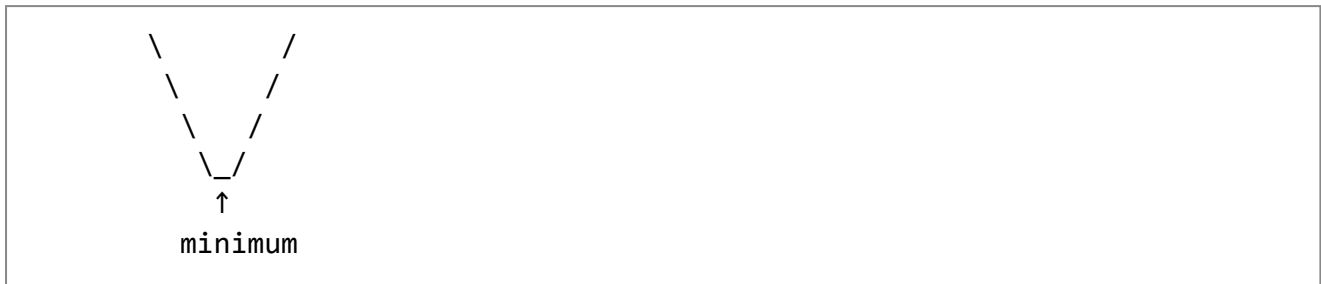
```
START
↓
Current parameter
↓
Calculate loss
↓
Calculate gradient
↓
Take downhill step
↓
New parameter
↓
Calculate again
↓
Repeat
```

Eventually:

```
large error
↓
smaller error
↓
smaller error
↓
even smaller error
```

15. Step too large

Suppose the bottom is here:



If your steps are too large:



you might repeatedly jump from one side of the minimum to the other.

For example:

```
10
↓
2
↓
8
↓
3
↓
7
...
```

The model can have difficulty settling near the best value.

In some situations, very large steps can even make the error worse and worse.

So:

```
learning rate too large
      ↓
updates may overshoot
```

16. Step too small

Now imagine extremely tiny steps:

```
10
↓
9.99
```

↓
9.98
↓
9.97
↓
...

The model may be moving in the correct direction, but progress can be very slow.

So:

learning rate too small
↓
stable but potentially slow learning

A useful intuition is:

Too large
→ may jump around or overshoot

Too small
→ may take many updates

Reasonable size
→ gradual progress toward lower loss

17. Connection to future Linear Regression

Soon, when you study **Linear Regression**, you might have a line:

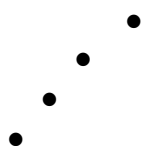
prediction = weight × input + bias

The model needs good values for:

weight
bias

Initially, its line might make poor predictions.

Actual data:



Poor model:

The model calculates an error.

Then gradient-based optimization can adjust its parameters:

```
wrong weight/bias
↓
calculate error
↓
calculate gradients
↓
adjust weight/bias
↓
new predictions
↓
smaller error
```

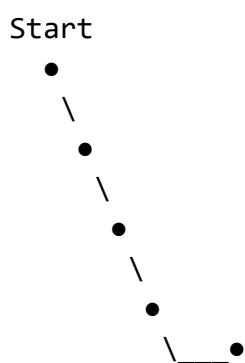
After repeated updates, the fitted line can improve.

Gradient-based optimization is also important in many other models, especially **neural networks**.

You do not need their mathematics yet.

18. Expected conceptual result

Starting from a parameter far from the low-error region:



you should expect repeated sensible updates to move approximately like:

```
high error
↓
lower error
↓
```

lower error
↓
near a minimum

So the main idea of Day 97 is:

Gradient
↓
tells us uphill direction

Gradient descent
↓
moves opposite the gradient

Learning rate
↓
controls step size

Repeated updates
↓
can gradually reduce model error

Or in one sentence:

Gradient descent repeatedly uses gradient information to make small parameter changes that aim to reduce a model's loss.

19. Hint only

For your exercise:

starting $w = 10$
target low-error area ≈ 4

Do not jump straight from:

$10 \rightarrow 4$

Instead, imagine a reasonable sequence such as:

10
↓
some smaller value
↓
another smaller value
↓

```
closer to 4
↓
...
```

At every step ask:

| Is my new parameter moving toward lower error?

And remember the central rule:

```
parameter
=
parameter - small_step × gradient
```

You only need to understand **direction + small repeated steps** today; the exact mathematics of optimizing real ML models comes later.